CYBER SECURITY INTERNSHIP | WEEK 9

SOC Introduction and Setup

Network intrusion detection with Snort 3 on Ubuntu Server

Organization	DG Interns Hub
Module	Security Operations Center (SOC)
Topic	SOC Introduction and Setup Using Snort IDS
Intern	Arumugapandi K
Intern ID	DG/SEPTEMBER/CYBER/177
Practical performed	3 November 2026
Submission deadline	5 November 2026

Practical Report

1. Executive Summary

This report documents the Week 9 practical task of the Cyber Security Internship at DG Interns Hub. The week introduces the Security Operations Center (SOC) and asks the intern to build a small network monitoring setup with an Ubuntu Server and the Snort intrusion detection system (IDS).

Objectives

- Understand what a SOC is, how L1, L2 and L3 analysts work, and how SIEM, IDS and IPS differ.
- Install Ubuntu Server in a virtual machine and configure its hostname and network.
- Install Snort 3 and verify the installation and configuration.
- Write custom Snort rules to detect ICMP ping traffic and HTTP GET requests.
- Generate real traffic, capture the Snort alerts and keep screenshots as evidence.

Practical work performed

An Ubuntu Server 26.04.1 LTS virtual machine (hostname `soc-server`) was created in Oracle VirtualBox and placed on a lab network with a Kali Linux machine. Snort was not available in the Ubuntu package repositories, so Snort++ 3.12.4.0 was built from source together with LibDAQ 3.0.28.0 and installed under `/usr/local/snort`. The configuration was validated with 0 warnings. Two custom rules were written in `local.rules`: SID 1000001 for ICMP echo requests and SID 1000002 for HTTP GET requests to port 80.

Key outcomes

- The ICMP rule produced alerts for pings from Kali (192.168.100.4) to Ubuntu (192.168.100.3). The captured Snort output lists 25 alerts.
- The HTTP rule (SID 1000002, revision 2) produced an alert for a GET request from 192.168.100.4:33958 to the Python web server on 192.168.100.3:80, which answered with HTTP 200.
- A VirtualBox host-only adapter error and a Snort fatal-error message were met during the lab and are described in Section 13.

Task coverage

Task	What was done	Report section	Figures
Task 1	SOC basics: definition, L1/L2/L3 roles, SIEM vs IDS vs IPS	Sections 2, 3, 4	-
Task 2	VirtualBox VM, Ubuntu Server, hostname, network, SSH	Sections 5, 6	1 to 7
Task 3	Snort build, installation check, configuration test	Sections 7, 8	8 to 14
Task 4	Snort rule structure, ICMP rule, HTTP rule	Section 9	15, 16
Task 5	ICMP and HTTP tests, alert capture	Sections 10, 11, 12	17 to 19

2. Introduction to SOC

A Security Operations Center (SOC) is a team that watches an organization's computers, servers and network all day and night. Its job is to find cyber attacks early and stop them before they cause serious damage. A simple way to picture it is a control room with CCTV cameras, except that the cameras watch the network instead of a building.

Purpose and importance

Attacks can happen at any time, including at night or on holidays. If an attack is found late, the loss is usually bigger. A SOC reduces this risk by watching logs and network traffic continuously and looking for strange or suspicious activity. When an alert appears, the team checks whether it is a real attack or a false alarm. For a real attack they respond by stopping it, cleaning the affected system and helping it recover. After each incident a report is written and the defences are improved so that the same problem does not repeat.

People, process and technology

Part	Meaning in a SOC
People	The security analysts who watch alerts, investigate incidents and respond.
Process	The agreed steps to follow when an alert comes in, such as playbooks for triage, escalation and reporting.
Technology	The tools that collect and analyse security data: SIEM, IDS, IPS and firewalls.

How SOC monitoring supports an organization

- **Early detection:** continuous monitoring finds suspicious activity before it spreads.
- **Faster response:** a defined process lets the team contain and recover quickly.
- **Learning:** every incident ends with a report, so detection rules and controls get better over time.

This week's lab is a small version of that idea: Snort watches the traffic on a server and raises an alert when a rule matches, which is the first step an L1 analyst would see in a real SOC.

3. SOC Analyst Roles

A SOC team is divided into levels. Easier work goes to the first level and harder work goes to the higher levels, so the most experienced people can focus on the serious problems. An easy way to remember the levels is: **L1 finds it, L2 fixes it, and L3 hunts and improves.**

Level	Role	Main responsibilities	Escalates to
L1	Triage analyst (entry level)	Watches alerts and dashboards, decides whether each alert is real or false, and follows ready-made playbooks.	L2, when an alert is serious or unclear
L2	Incident responder	Investigates escalated alerts, finds which systems are affected and how big the problem is, uses threat information such as known bad IP addresses, and contains the attack (for example by blocking an IP or isolating an infected computer).	L3, for the biggest or most difficult incidents
L3	Threat hunter / senior analyst	Handles major incidents, searches for hidden threats that the tools missed, and improves detection rules and security tools.	Management and tool owners

Alert escalation and investigation workflow

1. A detection tool (for example Snort) raises an alert.
2. The L1 analyst reviews it using the playbook and closes it if it is a false alarm.
3. If the alert is real or uncertain, L1 escalates it to L2 with the evidence collected so far.
4. L2 investigates the scope, confirms the attack with threat information and contains it.
5. Complex cases go to L3, who also hunt for related activity and tune the rules.
6. A report is written and the lessons are used to improve detection.

4. SIEM, IDS and IPS

These three technologies are often mentioned together, but each has a different job.

Definitions

SIEM (Security Information and Event Management) collects logs from many devices such as servers, firewalls and applications, keeps them in one place, compares them together and raises an alert when something looks suspicious. It gives the big picture of the network. Examples: Splunk, Wazuh and the ELK Stack.

IDS (Intrusion Detection System) watches network traffic and raises an alert when it sees an attack pattern. It only detects and reports; it does not stop the attack. It sits on the side of the network and checks a copy of the traffic. Snort in detection mode is an IDS, and this is how it was used in this lab.

IPS (Intrusion Prevention System) is similar to an IDS but sits in the middle of the traffic path, so it can detect an attack and also block it automatically. Examples: Suricata, and Snort in inline mode.

Comparison

Aspect	SIEM	IDS	IPS
Main function	Collects and correlates logs	Detects attacks	Detects and blocks attacks

Aspect	SIEM	IDS	IPS
Data used	Logs from many sources	Network traffic (a copy)	Live network traffic
Position	Central log platform	Beside the traffic path	Inline, in the traffic path
Action taken	Alerts and dashboards	Alert only	Alert and block
Examples	Splunk, Wazuh, ELK Stack	Snort (detection mode)	Suricata, Snort (inline)

How they work together

IDS and IPS send their alerts to the SIEM. The SIEM joins those alerts with other logs, so the analyst can understand the full story of an attack instead of looking at one alert alone. In this lab only the IDS part was built: Snort raised alerts on the Ubuntu server. In a real SOC these alerts would flow into a SIEM and an L1 analyst would check them first.

5. Lab Environment and Architecture

Objective: build a small, isolated monitoring lab where one machine generates traffic and another machine watches it.

Components

Component	Details	Role
Virtualization	Oracle VirtualBox	Runs both virtual machines
Ubuntu SOC server	Ubuntu Server 26.04.1 LTS, hostname `soc-server`, VM name SOC_SERVER	Runs Snort and the test web server
Kali Linux	Kali VM	Generates ICMP and HTTP test traffic
Lab network	VirtualBox NAT Network SOC-LAB-NET, 192.168.100.0/24	Connects the two machines
IDS	Snort++ 3.12.4.0, passive mode	Inspects traffic and raises alerts

Machine	Interface	IP address	Purpose
Ubuntu SOC server	enp0s8	192.168.100.3	Lab network; interface monitored by Snort
Kali Linux	eth1	192.168.100.4	Source of test traffic
Ubuntu SOC server	enp0s3	10.0.2.15	VirtualBox NAT adapter (Internet, SSH)

The NAT Network name and the Kali interface name come from the lab design; the VirtualBox network dialog and the Kali terminal were not captured. The Ubuntu addresses and the Kali source address are confirmed by Figures 2, 6 and 17.

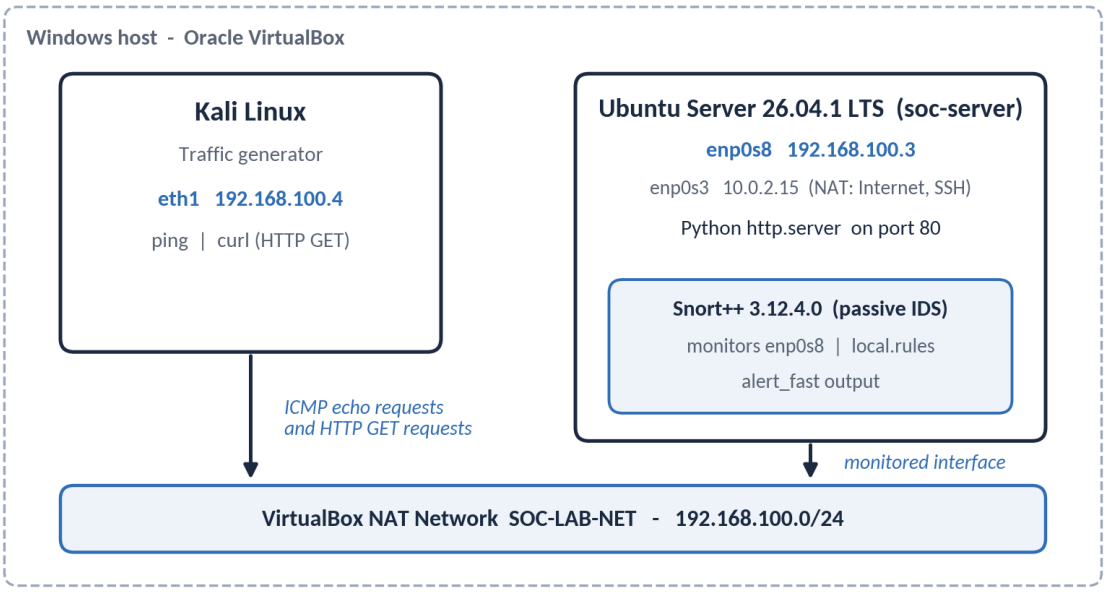


Figure 1. *Lab network architecture. Snort monitors interface enp0s8 on the Ubuntu server.*

Snort monitoring location. Snort runs on the Ubuntu server itself and listens passively on `enp0s8`. It therefore sees traffic that reaches or leaves the server on the lab network, such as pings and web requests coming from Kali. Figure 2 shows the final interface state.

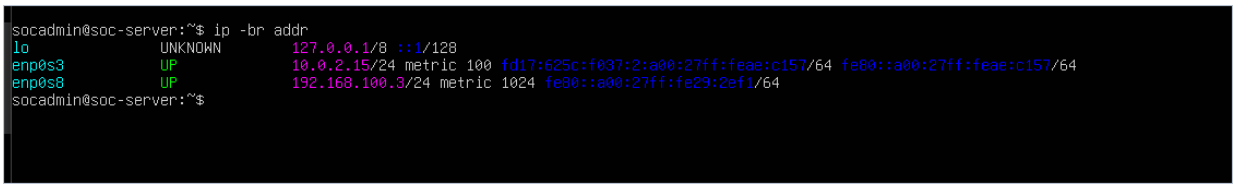


Figure 2. *Ubuntu interfaces: enp0s3 (10.0.2.15/24) and enp0s8 (192.168.100.3/24), both UP.*

6. Ubuntu Server Setup

Objective: create the Ubuntu server, name it, give it network access and confirm remote access.

Virtual machine and installation

A new virtual machine named SOC_SERVER was created in VirtualBox using the `ubuntu-26.04.1-live-server-amd64.iso` image. VirtualBox detected the OS type as Ubuntu (64-bit) and the unattended installation option was used. The user name `socadmin` and the installer host name SOC were entered on the next screen. The VM was given 4096 MB of base memory and 2 processors (visible in Figure 21).

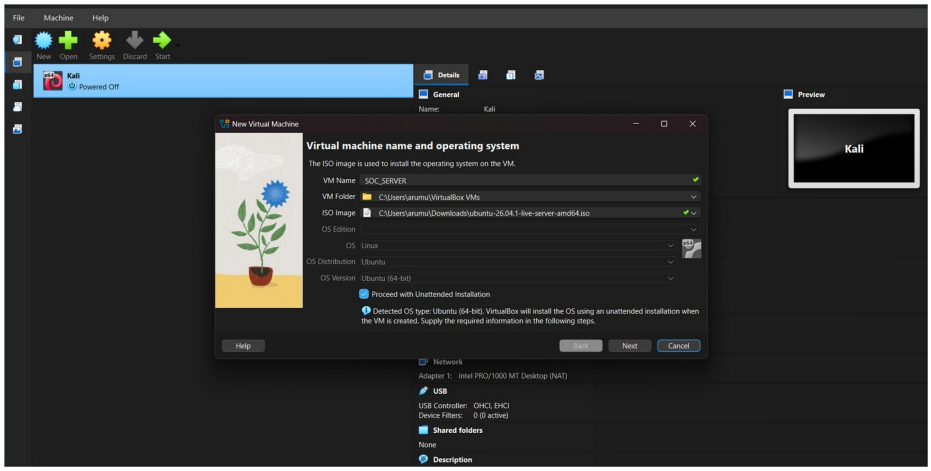


Figure 3. *VirtualBox New Virtual Machine wizard: VM name SOC_SERVER and the Ubuntu 26.04.1 server ISO.*

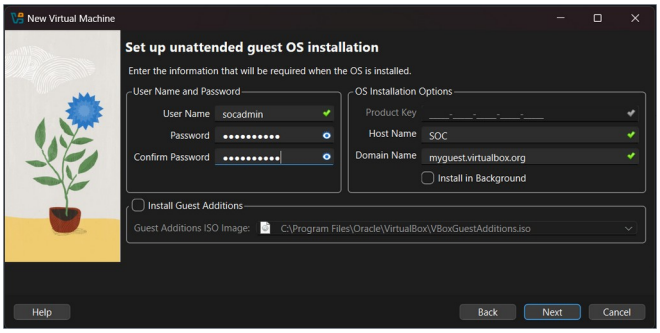


Figure 4. *Unattended installation settings: user name socadmin and host name SOC (password hidden).*

Hostname configuration

The installer host name was SOC. It was changed to `soc-server` with `sudo hostnamectl set-hostname soc-server`, and `hostnamectl` confirmed the static hostname, Ubuntu 26.04.1 LTS, kernel `7.0.0-42-generic` and Oracle virtualization. The prompt still shows SOC in Figure 5 because the open session had not been refreshed; later screenshots show `soc-server`.

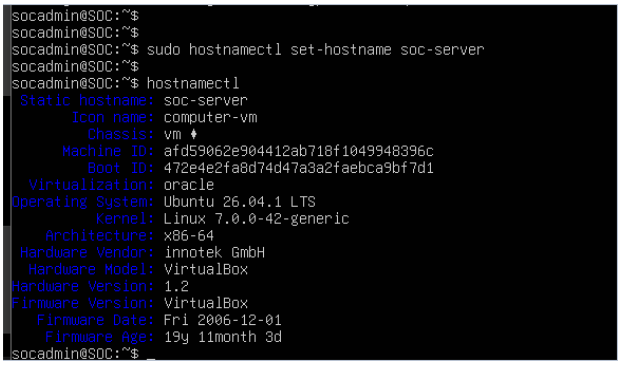


Figure 5. *Hostname set to soc-server and verified with hostnamectl.*

Network configuration and connectivity

At this stage the server had one adapter, `enp0s3`, with the address 10.0.2.15/24 and a default route through 10.0.2.2 (the VirtualBox NAT gateway). Internet access was checked by pinging 8.8.8.8 and `google.com`; both tests returned 4 of 4 replies with 0% packet loss.

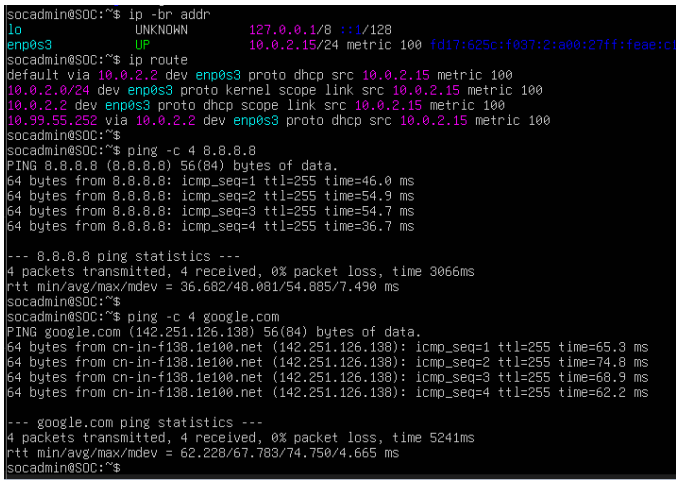


Figure 6. *Network configuration (ip -br addr, ip route) and ping tests to 8.8.8.8 and google.com.*

SSH access

The OpenSSH service was checked with `sudo systemctl status ssh`. It was enabled and active (running), listening on port 22, and the log shows an accepted password login for `socadmin` from 10.0.2.2, the host side of the NAT connection. This allowed the server to be managed from a normal terminal window on the host.

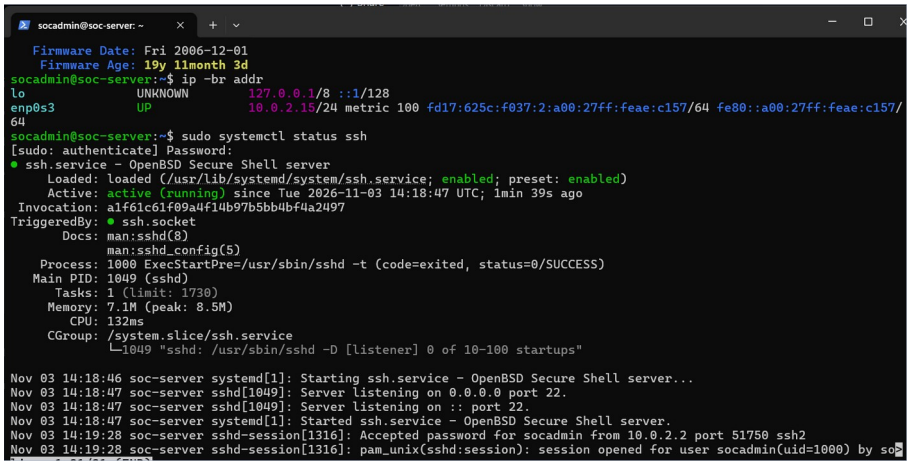


Figure 7. *SSH service status: active (running) and an accepted session for socadmin.*

Evidence note: system update commands (`apt update` and `apt upgrade`) were not captured in the

screenshots supplied for this week, so no update output is shown. Figure 8 shows the Ubuntu repository configuration that the server used.

7. Snort Installation

What is Snort and why it was selected

Snort is a free, open-source network intrusion detection and prevention system. It reads network packets, compares them with a set of rules and raises an alert when a rule matches. Snort 3 (also called Snort++) is the current generation. It is configured with Lua files and uses the DAQ (Data Acquisition) library to read packets. Snort was selected because it is the IDS named in the assignment, it is free, and its rule language is easy to read and write, which makes it good for learning how detection works.

Installation approach

Objective: install Snort 3 on Ubuntu Server 26.04.1.

First, the package repositories were checked. `apt-cache policy snort` returned "Unable to locate package snort", and `apt-cache search snort` listed only related packages such as `libdaq-dev`, `libdaq2t64` and `suricata`. The repositories in use were the `resolute`, `resolute-updates`, `resolute-backports` and `resolute-security` suites of the Ubuntu archive. Because Snort could not be installed with `apt`, it was built from source.



Figure 8. *Package check: "Unable to locate package snort", Ubuntu release information and repository sources.*

Build tools and LibDAQ

The build tools were checked first: GCC 15.2.0, CMake 4.2.3 and Git 2.53.0. LibDAQ was then built in `~/snort-src/libdaq`. The configure summary shows which DAQ modules were enabled: AFPacket, BPF, Dump, FST, PCAP, Savefile, Trace and GWLB were built, while Divert, netmap and NFQ were not.

```
socadmin@soc-server:~$
socadmin@soc-server:~$ cmake --version
gcc --version
git --version
cmake version 4.2.3

CMake suite maintained and supported by Kitware (kitware.com/cmake).
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

git version 2.53.0
socadmin@soc-server:~$
```

Figure 9. *Build tool versions: cmake 4.2.3, gcc 15.2.0 and git 2.53.0.*

```
code_coverage_enabled:  no
code_coverage_cppflags:
code_coverage_cflags:
code_coverage_ldflags:

Build AFPacket DAQ module.. : yes
Build BPF DAQ module....... : yes
Build Divert DAQ module.... : no
Build Dump DAQ module...... : yes
Build FST DAQ module....... : yes
Build netmap DAQ module.... : no
Build NFQ DAQ module....... : no
Build PCAP DAQ module...... : yes
Build Savefile DAQ module.. : yes
Build Trace DAQ module..... : yes
Build GWLB DAQ module...... : yes

admin@soc-server:~/snort-src/libdaq$
```

Figure 10. *LibDAQ configure summary showing the DAQ modules selected for the build.*

After the build, `sudo ldconfig` refreshed the library cache. `pkg-config --modversion libdaq` reported **3.0.28.0**, and `/usr/local/lib/daq` contained the built modules (`daq_afpacket`, `daq_bpf`, `daq_dump`, `daq_fst`, `daq_gwlb`, `daq_pcap`, `daq_savefile` and `daq_trace`).

```
make[2]: Leaving directory '/home/socadmin/snort-src/libdaq'
make[1]: Leaving directory '/home/socadmin/snort-src/libdaq'
socadmin@soc-server:~/snort-src/libdaq$ sudo ldconfig
socadmin@soc-server:~/snort-src/libdaq$ pkg-config --modversion libdaq
3.0.28.0
socadmin@soc-server:~/snort-src/libdaq$ ls /usr/local/lib/daq
daq_afpacket.la  daq_bpf.la  daq_dump.la  daq_fst.la  daq_gwlb.la  daq_pcap.la  daq_savefile.la  daq_trace.la
daq_afpacket.so  daq_bpf.so  daq_dump.so  daq_fst.so  daq_gwlb.so  daq_pcap.so  daq_savefile.so  daq_trace.so
socadmin@soc-server:~/snort-src/libdaq$
```

Figure 11. *LibDAQ verification: ldconfig, version 3.0.28.0 and the installed DAQ modules.*

Snort build and version verification

Snort 3 was built from `~/snort-src/snort3/build` and installed under the prefix `/usr/local/snort`. Running `/usr/local/snort/bin/snort -V` confirmed **Snort++ 3.12.4.0** using DAQ 3.0.28.0, together with libpcap 1.10.6, LuaJIT 2.1.1761786044, LZMA 5.8.3, OpenSSL 3.5.5, PCRE2 10.46 and ZLIB 1.3.1.

```
socadmin@soc-server:~/snort-src/snort3/build$
socadmin@soc-server:~/snort-src/snort3/build$ /usr/local/snort/bin/snort -V

   ,,_     -*> Snort++ <*-
  o"  )~   Version 3.12.4.0
   ''''    By Martin Roesch & The Snort Team
           http://snort.org/contact#team
           Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using DAQ version 3.0.28.0
           Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
           Using LuaJIT version 2.1.1761786044
           Using LZMA version 5.8.3
           Using OpenSSL 3.5.5 27 Jan 2026
           Using PCRE2 version 10.46 2025-08-27
           Using ZLIB version 1.3.1

socadmin@soc-server:~/snort-src/snort3/build$
```

Figure 12. *Snort version output: Snort++ 3.12.4.0 with DAQ 3.0.28.0.*

8. Snort Configuration

Objective: confirm that the installed Snort starts and accepts its configuration.

Main configuration file and DAQ

The main configuration file is `/usr/local/snort/etc/snort/snort.lua`. It includes `snort_defaults.lua` and sets `HOME_NET` and `EXTERNAL_NET`; in this lab both are `any`. The same directory holds the default Lua files and the sample rule files `file_magic.rules` and `sensitive_data.rules`.

The DAQ layer sits between the network interface and Snort. The command `snort --daq-list` printed the available modules (for example `afpacket`, `bpf`, `dump` and `fst`). The start-up output confirms that the **pcap DAQ is configured to passive**, so Snort only inspects traffic and never blocks it.

```
socadmin@soc-server:~/snort-src/snort3/build$ grep -nE 'RULE_PATH|ips =|rules|include' \
/usr/local/snort/etc/snort/snort.lua
30:include 'snort_defaults.lua'
92:-- see file_magic.rules for file id rules
93:file_inspect = { rules_file = 'file_magic.rules' }
102:    -- appid requires this to use appids in rules
188:ips =
191:    --enable_builtin_rules = true,
193:    -- use include for rules files; be sure to set your path
194:    -- note that rules files can include other rules files
231:    -- reduce the number of events logged for some rules
280:    include(tweaks .. '.lua')
socadmin@soc-server:~/snort-src/snort3/build$ grep -nE 'HOME_NET|EXTERNAL_NET' \
/usr/local/snort/etc/snort/snort.lua
22:-- HOME_NET and EXTERNAL_NET must be set now
24:HOME_NET = 'any'
28:EXTERNAL_NET = 'any'
socadmin@soc-server:~/snort-src/snort3/build$ ls -la /usr/local/snort/etc/snort/
total 160
drwxr-xr-x 2 root root  4096 Nov  3 14:43 .
drwxr-xr-x 3 root root  4096 Nov  3 14:43 ..
-rw-r--r-- 1 root root   590 Nov  3 14:29 balanced.lua
-rw-r--r-- 1 root root   616 Nov  3 14:29 connectivity.lua
-rw-r--r-- 1 root root 45258 Nov  3 14:29 file_magic.rules
-rw-r--r-- 1 root root   410 Nov  3 14:29 inline.lua
-rw-r--r-- 1 root root  1123 Nov  3 14:29 max_detect.lua
-rw-r--r-- 1 root root   814 Nov  3 14:29 security.lua
-rw-r--r-- 1 root root  5426 Nov  3 14:29 sensitive_data.rules
-rw-r--r-- 1 root root  9241 Nov  3 14:29 snort.lua
-rw-r--r-- 1 root root 59515 Nov  3 14:29 snort_defaults.lua
-rw-r--r-- 1 root root   791 Nov  3 14:29 talos.lua
socadmin@soc-server:~/snort-src/snort3/build$
```

Figure 13. *Settings in snort.lua (include, ips, HOME_NET, EXTERNAL_NET) and the Snort configuration directory.*

Configuration validation

The configuration test finished with the message "Snort successfully validated the configuration (with 0 warnings)". The command that started this test is above the visible area of the screenshot. The same screen also shows the `--daq-list` command that followed.

```
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~   Snort exiting
socadmin@soc-server:~/snort-src/snort3/build$ sudo /usr/local/snort/bin/snort \
```

Figure 14. *Snort configuration validated with 0 warnings; the next command shown is snort --daq-list.*

Rules directory and monitoring interface

A directory for custom rules was created at `/usr/local/snort/etc/snort/rules` and the file `local.rules` was edited there (Figure 15). During the detection tests Snort was attached to `enp0s8`; the start-up output shows `++ [0] enp0s8` (Figure 17). Alerts are printed to the terminal in the fast alert format.

9. Custom Snort Rules

Objective: write one rule for ICMP ping and one rule for HTTP GET requests.

Rule structure

A Snort rule has two parts. The **rule header** states what to do and which traffic to look at. The **rule options**, written in brackets, describe what to match and how to label the alert.

Component	Meaning
Action	What Snort does on a match. `alert` writes an alert.
Protocol	The protocol to inspect, for example `icmp` or `tcp`.
Addresses and ports	Source address and port, direction `->`, then destination address and port. `any` matches everything.
`msg`	The text printed in the alert.
`sid`	Unique rule ID. Numbers from 1000000 upwards are used for local rules.
`rev`	Revision number, increased when a rule is changed.

The two rules

```
alert icmp any any -> any any (msg:"SOC ALERT - ICMP Ping Detected"; itype:8; sid:1000001;
rev:1;)


alert tcp any any -> any 80 (msg:"SOC ALERT - HTTP GET Request Detected";
flow:to_server,established; http_method; content:"GET"; sid:1000002; rev:2;)
```

ICMP rule (SID 1000001, rev 1). It matches any ICMP packet where `itype:8`, which is an ICMP Echo Request (a ping). Echo replies have type 0 and are not matched, so each ping produces one alert, in the direction Kali to Ubuntu.

HTTP rule (SID 1000002, rev 2). It looks at TCP traffic going to port 80. `flow:to_server,established` limits the match to packets sent by the client in an established connection. `http_method` tells Snort to look only at the HTTP method field, and `content:"GET"` matches requests that use the GET method. The destination is `any`, so the rule is not tied to one server; in this lab only the Python web server on 192.168.100.3 was listening on port 80.

Evidence

Figure 15 shows the rules directory being created and `local.rules` being written and displayed. The HTTP rule in that listing is an earlier version: it is marked `rev:1` and uses `content:"GET "` with a positional option whose name is partly cut off at the screenshot edge. Figure 16 shows the final file, which is the version that produced the HTTP alert (the alert in Figure 19 carries revision 2).

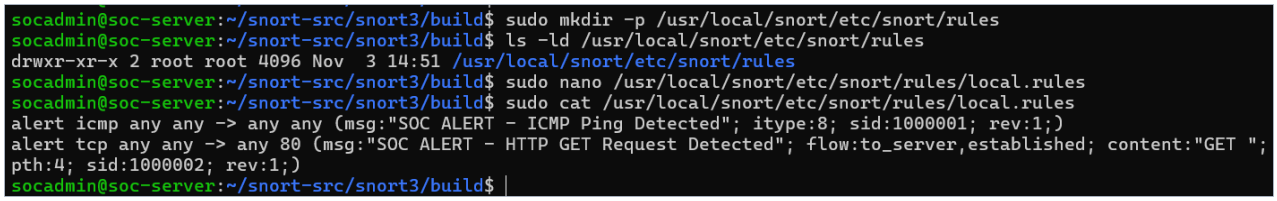


Figure 15. *Creation of the rules directory and the first listing of local.rules (HTTP rule at rev 1).*



Figure 16. *Final local.rules: ICMP rule SID 1000001 rev 1 and HTTP GET rule SID 1000002 rev 2.*

10. ICMP Detection Experiment

Objective: show that Snort detects ping traffic with rule SID 1000001.

Environment: Snort 3.12.4.0 on `soc-server`, monitoring `enp0s8` (192.168.100.3) in passive mode. Kali Linux on the same lab network with the address 192.168.100.4.

Procedure: Snort was started on `enp0s8` with the custom rules loaded. Ping requests were then sent from Kali to 192.168.100.3. The Snort terminal shows its start-up messages ("pcap DAQ configured to passive", "Commencing packet processing" and "++ [0] enp0s8") followed by the alerts.

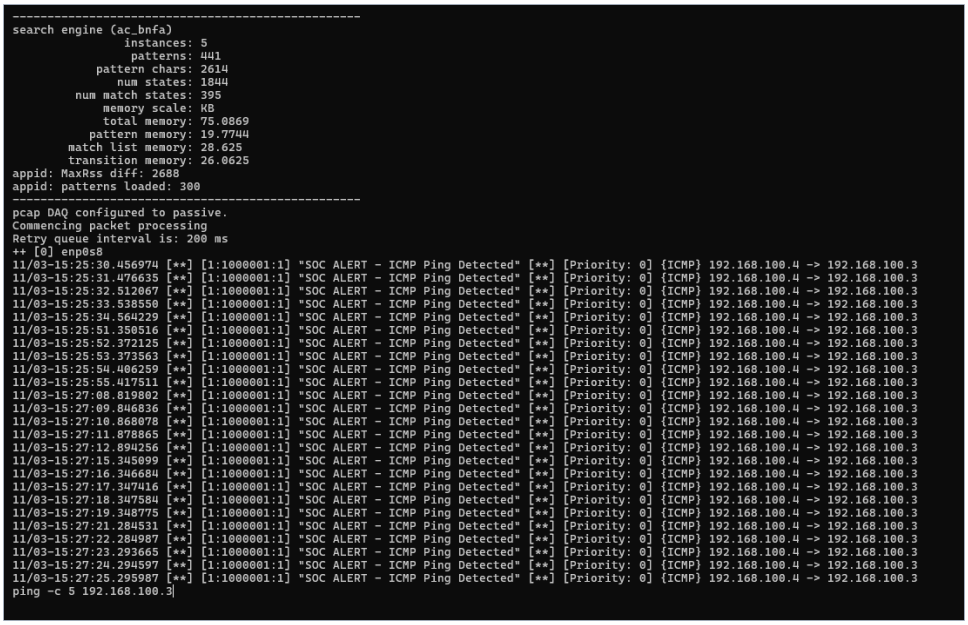


Figure 17. *Snort alerts for ICMP Echo Requests from 192.168.100.4 to 192.168.100.3 (SID 1000001).*

Result: every alert has the form `[1:1000001:1] "SOC ALERT - ICMP Ping Detected" [Priority: 0] {ICMP} 192.168.100.4 -> 192.168.100.3`. The screenshot lists 25 alerts in five groups of five, between 15:25:30 and 15:27:25 (server time, UTC). Within each group the alerts are about one second apart, which fits a ping run of five packets. The text `ping -c 5 192.168.100.3` at the bottom of the terminal is typed text left on the prompt line, not Snort output.

Technical explanation: the alert number `1:1000001:1` is the generator ID, the rule SID and the revision. The source address is the Kali machine that sent the Echo Request and the destination is the monitored address of the Ubuntu server. The Echo Replies travel in the opposite direction with ICMP type 0, so they do not match `itype:8`. The priority is 0 because the rule has no priority or class type set.

11. HTTP Detection Experiment

Objective: show that Snort detects an HTTP GET request with rule SID 1000002.

Environment: a small web server was started on the Ubuntu server with Python, bound to the lab address on port 80. Kali sent GET requests to it with `curl`.

Local HTTP server

The server was started with `sudo python3 -m http.server 80 --bind 192.168.100.3`. It printed "Serving HTTP on 192.168.100.3 port 80". Its log shows two requests from 192.168.100.4, both `GET / HTTP/1.1`, at 15:29:32 and 15:29:50 on 03 Nov 2026, and both were answered with status 200.

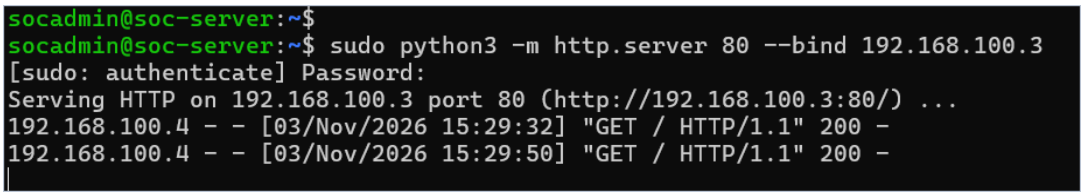


Figure 18. *Python HTTP server on 192.168.100.3:80 answering GET requests from 192.168.100.4 with HTTP 200.*

Snort detection

At the same time, Snort printed the following alert, which is the last line of the capture:

```
11/03-15:29:50.317497 [**] [1:1000002:2] "SOC ALERT - HTTP GET Request Detected" [**]
[Priority: 0] {TCP} 192.168.100.4:33958 -> 192.168.100.3:80
```

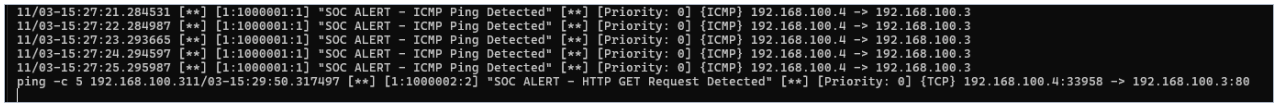


Figure 19. *Snort HTTP GET alert (SID 1000002, rev 2) as the last line of the capture, below the final ICMP alerts. The text "ping -c 5 192.168.100.3" at the start of the HTTP line is typed text on the terminal line.*

Alert analysis

The alert time, 15:29:50, matches the second request in the web server log. The source is Kali (192.168.100.4) using the client port 33958, and the destination is the web server (192.168.100.3) on port 80. The alert carries revision 2, which confirms that the final version of the rule was the one running. Only the HTTP rule fired for this traffic; the ICMP rule is not involved because the packets are TCP.

12. Results and Observations

Test	Traffic	Rule	Result
ICMP	Ping from Kali (192.168.100.4) to Ubuntu (192.168.100.3)	SID 1000001, rev 1	Detected. 25 alerts shown, all `{ICMP}` `192.168.100.4 -> 192.168.100.3`
HTTP	curl GET from Kali to Python server on 192.168.100.3:80	SID 1000002, rev 2	Detected. Alert at 15:29:50, `{TCP}` `192.168.100.4:33958 -> 192.168.100.3:80`; server replied HTTP 200

What the results show

- The full detection chain worked: traffic from a second machine reached the monitored interface, Snort matched it against the custom rules and printed alerts with the correct source and destination.
- Both rules matched real traffic, so the rule headers and options were written correctly.
- Snort ran as a passive IDS. It reported the traffic but did not block it, which is the difference between an IDS and an IPS described in Section 4.

Observations and limits

- The two HTTP requests in the server log are at 15:29:32 and 15:29:50, but the captured Snort screen shows an alert only for the second one. The evidence does not record why, so this report does not give a cause.
- Both alerts have priority 0 because no priority or class type was added to the rules.

- HOME_NET and EXTERNAL_NET are both any and the HTTP rule uses any as destination, so the rules would match matching traffic to any host seen on the monitored interface.
- The tests used one server and one client in a closed lab, so the results show that the rules work, not how Snort performs under heavy traffic. No performance measurements were taken.

13. Challenges and Troubleshooting

Issue	What the evidence shows and how it was handled
Ubuntu package availability	`apt-cache policy snort` returned "Unable to locate package snort" on Ubuntu 26.04.1 (Figure 8). Snort was installed by building it from source instead.
Snort source build	LibDAQ had to be built and registered first (`ldconfig`, `pkg-config` version 3.0.28.0, modules in `/usr/local/lib/daq`). Snort 3 was then built in its `build` directory and installed to `/usr/local/snort`, and `snort -V` confirmed it works (Figures 9 to 12).
VirtualBox networking	Both VMs failed to start with the error `VERR_INTNET_FLT_IF_NOT_FOUND` for the host-only adapter, and both showed as Aborted (Figure 21). The lab was then run on the NAT Network `SOC-LAB-NET`. The VirtualBox dialog for this change was not captured, but Figure 2 shows the working result: `enp0s8` with 192.168.100.3/24.
Snort fatal error	One validation run ended with "FATAL: see prior 1 errors (0 warnings)" (Figure 20). The line with the actual error is above the visible area, so the cause cannot be confirmed from the evidence. The same run loaded 221 rules (the 219 default rules plus the 2 custom rules). Later runs on `enp0s8` started normally and produced alerts.
HTTP rule revision	The first listing of `local.rules` shows the HTTP rule at rev 1 with a raw `content:"GET "` match (Figure 15). The final file and the alert show rev 2, which matches GET inside the `http_method` buffer (Figures 16 and 19). The reason for the change was not recorded; only one version was active during the HTTP test.

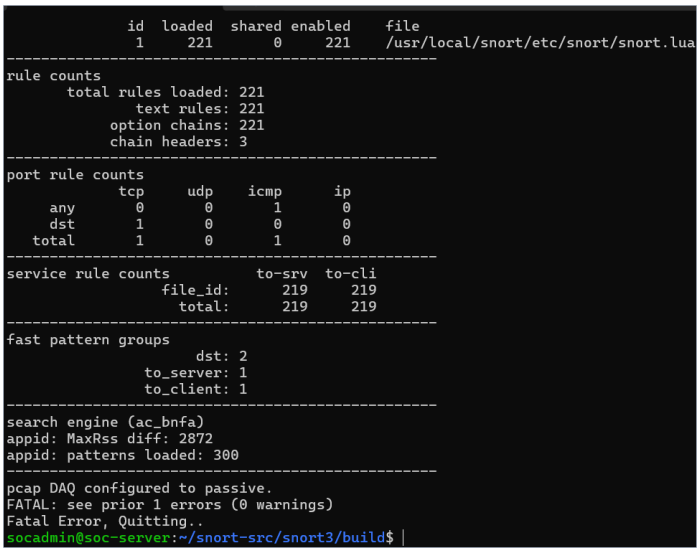


Figure 20. *Snort run ending with "FATAL: see prior 1 errors"; 221 rules were loaded (ICMP rule and one TCP destination-port rule included).*

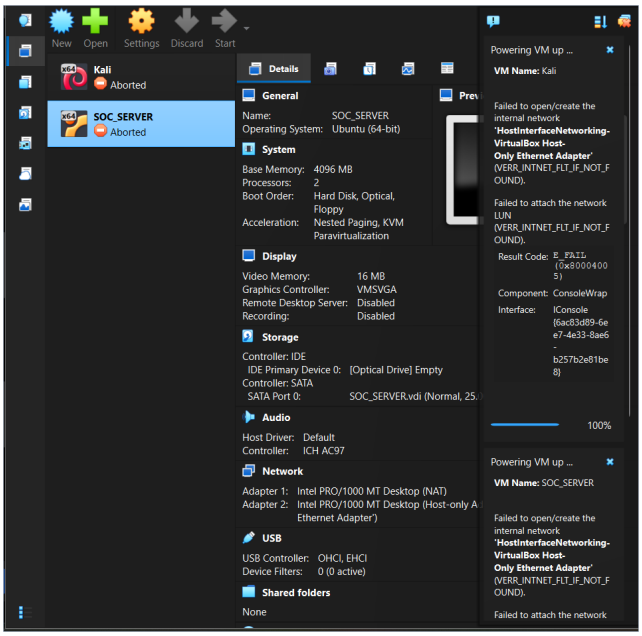


Figure 21. *VirtualBox error for the host-only adapter (VERR_INTNET_FLT_IF_NOT_FOUND); both VMs aborted.*

14. Learning Outcomes and Conclusion

What was learned

- **SOC concepts:** what a SOC is, how L1, L2 and L3 work together, and how SIEM, IDS and IPS differ and connect.
- **Linux administration:** creating a VM, setting a hostname, checking networking and SSH, and building software from source when no package exists.
- **Network monitoring:** using Snort on a chosen interface and reading the fast alert format (timestamp, GID:SID:REV, message, protocol, source and destination).
- **Custom rule creation:** writing rules with the right header and options, and understanding SID and revision numbers.
- **Alert verification:** matching Snort alerts with the traffic that caused them, for example comparing the web server log with the alert time.
- **Troubleshooting:** working through a missing package, a VirtualBox adapter error and a validation error while keeping honest notes about what was and was not proven.

Conclusion

The Week 9 lab met all five assignment tasks. An Ubuntu Server was prepared, Snort++ 3.12.4.0 was built and validated, and two custom rules detected real ICMP and HTTP GET traffic from Kali with the expected source and destination addresses. The work gave a clear, hands-on view of how an IDS produces the alerts that an L1 analyst would review in a SOC.

Future learning

- Forward Snort alerts to a SIEM such as Wazuh, Splunk or the ELK Stack.
- Narrow `HOME_NET` to 192.168.100.0/24 and add priorities or class types to the rules.
- Try Snort in inline mode as an IPS and compare it with passive detection.